Industrial Applications of Q-Learning: A Systematic Review

GUILLERMINA VIVAR-ESTUDILLO[1], MARIA ESTUDILLO-AYALA[1],
JOSE BELTRÁN-HERNÁNDEZ[1], MARIO IBARRA-MANZANO[2],
CARLOS LASTRE-DOMÍNGUEZ[3,*]

[1]Facultad de Sistemas Biológicos e Innovación Tecnológica,
Universidad Autónoma Benito Juárez de Oaxaca,
Av. Universidad S/N. Ex-Hacienda 5 Señores, Oaxaca de Juárez, Oax. C.P. 68120,
MEXICO

[2]Departament of Electronics Engineering,
Universidad de Guanajuato,
Carretera Salamanca - Valle de Santiago Km. 3.5 + 1.8, Salamanca, Gto, C.P. 36885,
MEXICO

[3]División de Estudios de Posgrado e Investigación,
Tecnológico Nacional de México/Instituto Tecnológico de Oaxaca,
Avenida Ing. Víctor Bravo Ahuja No. 125 Esquina Calzada Tecnológico,
Oaxaca de Juárez, Oax. C.P. 68030,
MEXICO

**Corresponding Author*

Abstract: - Artificial Intelligence (AI) couples various computational methods to replicate human learning and is crucial for addressing intricate problems. A significant aspect of AI is reinforcement learning (RL), which comprises algorithms that can resolve practical challenges, with Q-learning being a prominent example. Q-learning enhances the learning process, serving as a fundamental element of reinforcement learning, to discover optimal solutions without relying on a specific policy. As the demand for AI applications increases, Q-learning has gained traction in numerous sectors such as energy management, robotics, finance, game design, medicine, and logistics. This paper provides a concise overview of Q-learning applications across different industries, examining their main results, challenges, and limitations.

Key-Words: - Q-learning, off-policy, machine learning, reinforcement learning, artificial intelligence, systematic review.

Received: August 2, 2024. Revised: April 23, 2025. Accepted: May 22, 2025. Published: July 14, 2025.

1 Introduction

Artificial Intelligence (AI), a set of computational techniques designed to replicate human learning processes, has become essential for tackling complex challenges. Within the AI domain, Machine Learning (ML) stands out as a significant category that includes four primary types of learning: supervised learning (SL), unsupervised learning (UL), transfer learning (TL), and reinforcement learning (RL). SL and UL methods involve learning algorithms and are considered traditional due to their wide-ranging applications across various fields, [1], [2], [3], [4]. Notably, some of the most used algorithms in this context include Support Vector Machines (SVM), Decision Trees (DT), conventional Neural Networks (NN), k-nearest Neighbors (KNN), k-means clustering, and ensemble methods, [5], [6], [7], [8], [9], [10], [11], [12], [13], [14].

Another fundamental approach is deep learning (DL), a subcategory of Machine Learning (ML), [15], [16], [17], [18], [19], [20], [21], [22]. This category focuses on training neural networks at their internal layer level, which can be configured as convolutional neural networks (CNN), short-term memory networks (STMN) for time series,

generative adversarial networks (GAN), or residual neural networks (ResNet), among others. Recent studies show these techniques are vital for addressing various industrial challenges and problems, [23], [24]. In the literature, we find applications of supervised learning (SL), such as in robotics, where it controls robotic arms through electromyographic signals in real-time using techniques like Random Forest (RF), CNN networks, and Temporal Multi-Channel Transformers (TMC-T). In another study, Support Vector Machines (SVM) are applied to detect weeds and control sprayers in tobacco crops, while other research focuses on detecting anomalies in biomedical images and bio signals, [25], [26].

Deep learning techniques have increasingly been used in energy management, especially within distributed systems that incorporate alternative sources like photovoltaic panels and wind turbines. However, several challenges remain, including long algorithm training times, limitations in task specificity, and strict quality requirements in industrial production, [27]. Recently, Reinforcement Learning (RL) has gained significant attention due to its potential to solve complex problems across various industries, [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38]. RL focuses on developing a system that determines optimal actions based on rewards as it progresses toward achieving its learning objectives.

This paper presents a preliminary review of Q-learning applications across various industries, with a focus on strategies to enhance performance and address challenges. The research considers databases such as IEEE Xplore, Google Scholar, and PubMed. The findings indicate a growing interest in Q-learning, particularly in sectors like energy management, where it has contributed to optimizing energy quality and efficiency. However, the review also highlights limitations, including high computational complexity and slow algorithm convergence in real-world environments. Unlike existing reviews focusing on theoretical Q-Learning aspects or generic applications, this work provides an up-to-date comparative analysis (2022-2024) of Q-Learning variants in real-world industrial settings, identification of critical gaps such as sensor dependency in HRC and scalability in scheduling, and a reproducible benchmarking method with impact-based filters.

2 Reinforcement Learning: Q-Learning

A reinforcement learning process consists of considering a function that evaluates decision-making based on a set of random solutions, whose goal is to find the optimal solution or the best policy. This function is considered the agent, which will assign weights or rewards according to the actions performed in the process, [39], [40], [41], [42], [43]. This process can be visualized in Figure 1, which indicates an agent-environment interaction protocol where a $Q(s,a)$ function, represented in a Q-table, is updated as the agent's actions and states vary.

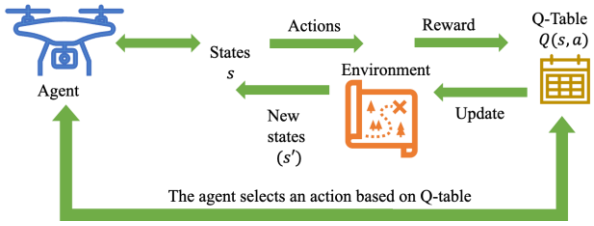

Fig. 1: Agent-environment iteration protocol

In reinforcement learning, there are two strategies: on-policy and off-policy. These approaches are associated with the classification of learning types in this area. Based on these approaches, there are two main methods: State-Action-Reward-State-Action (SARSA) and Q-Learning. Both approaches are based on an algorithm, as equation (1) indicates.

$$Q(s,a) \leftarrow (1-\propto)Q(s,a) \quad + \quad \beta, \qquad (1)$$

The difference is found with the variable Considering for the SARSA technique is represented by equation (2),

$$\beta_s = \propto [r' + \gamma\, Q(s',a')] \qquad (2)$$

Equation (3), known as Bellman's equation, associates the Q-learning technique.

$$\beta_Q = \propto \left[r + \gamma \max_{a'} Q(s',a')\right], \qquad (3)$$

Where the function is the value of the action in the state, the constant α is the learning rate that controls how much the value of in each iteration is updated, is the reward obtained by taking the action $Q\,(s,a)$ in the state is the discount factor that determines the importance of future rewards, it is the next state after taking the action, and it represents the maximum possible value for the next state, [44] $Q(s',a')$. Unlike the SARSA approach, the Q-learning technique aims to find an optimal

policy, i.e., the best action at any given time without the need to follow a defined policy, [45].

In the literature, there are different techniques and variants of Q-Learning within the context of RA, as can be seen in Figure 2.

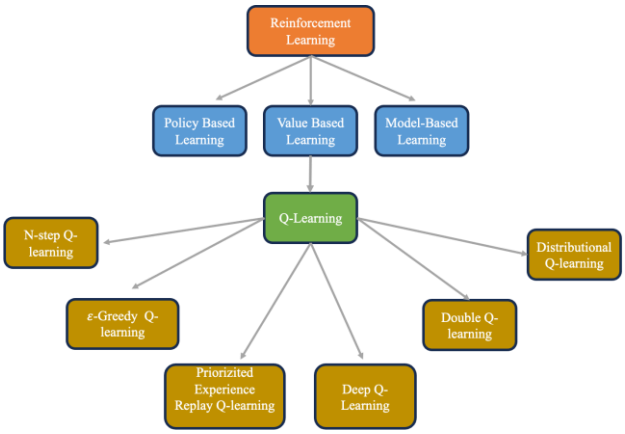

Fig. 2: Variants of Q-Learning

Research associated with different applications of Q-Learning in industry, among which it stands out the most, are applications in robotics, industrial process control, game theory, finance, energy management, medicine, and logistics.

2.1 Q-learning Approaches

2.1.1 ε-Greedy Q-learning

The ε-greedy method is a straightforward approach to balancing exploration and exploitation by randomly selecting between the two (see equation 4). In this method, "epsilon" represents the probability of choosing exploration. Most of the time, it favors exploitation, but there is a small chance of opting for exploration, [46],[47], [48].

$$a(t) = \begin{cases} \max Q(a) & p = 1 - \varepsilon \\ Q'(a) & p = \varepsilon \end{cases} \quad (4)$$

Where $\max Q(a)$ is the optimal action, $Q'(a)$ indicates a random action and p is the probability depending on epsilon greedy.

The algorithm proposed in [47] (see Algorithm 1), the authors implement an adaptive ε-greedy strategy for reinforcement learning, dynamically adjusting the exploration rate (ε) based on observed performance. It begins by initializing parameters, including a window size l for evaluation and a scaling factor f. During execution, the algorithm either explores by selecting random actions (with probability ε) or exploits by choosing the action with the highest Q-value.

Algorithm 1: Adaptive ε-Greedy

Input:
- Q: State-action value table
- $\varepsilon_{initial}$: Initial ε value
- l: Evaluation window
- f: Scaling factor

Begin:
1: $max_{prev} \leftarrow 0$
2: k ← 0
3: ε ← $\varepsilon_{initial}$
4:For each time step t **do:**
5: **if** rand() < ε **then:**
6: $A_t \leftarrow$ **random action**
7: $max_{curr} \leftarrow Q(A\_t)$
8: $k \leftarrow k + 1$
10: **if** $k == l$ **then:**
11: $\delta \leftarrow (max_{curr} - max_{prev}).f$
12: **if** $\delta > 0$ **then:**
13: $\varepsilon \leftarrow sigmoid(\delta)$
14: **else if** $\delta < 0$ **then:**
15: $\varepsilon \leftarrow 0.5$
16: **end if**
18: $max_{prev} \leftarrow max_{curr}$
19: k ← 0
20: **end if**
21: **else:**
22: $A_t \leftarrow \max_a Q(a)$
23: **end if**
24: end for
25: **return** $x = 1 / (1 + exp(-x))$

Every l step, it calculates a performance metric δ by comparing the current maximum Q-value (max_{curr}) with the previous one (max_{prev}), scaled by f. If δ is positive (indicating improvement), ε is updated using a sigmoid function to reduce exploration gradually. If δ is negative (suggesting performance decline), ε resets to 0.5 to encourage renewed exploration. This adaptive mechanism ensures a balance between exploration and exploitation, optimizing learning efficiency. The sigmoid function (defined at the end) smooths ε adjustments, preventing abrupt changes.

2.1.2 Prioritized Experience Replay (PER) Q-learning

The prioritized experience replay mechanism, although useful for decoupling temporally correlated experiences, has significant limitations as it relies on sequential storage and random sampling, [49]. This approach assumes that all experiences have equal value, leading to inefficient processing of uninformative samples and an imbalanced

distribution of critical events. Low-value experiences, such as those with minimal or redundant rewards, consume computational resources without significantly improving learning. Additionally, random sampling may overlook rare but important transitions, slowing model convergence and impacting accuracy. These limitations highlight the need for advanced approaches, such as PER which prioritize the most relevant samples to enhance learning efficiency and effectiveness, [49].

Algorithm 2: Double DQN with Proportional Prioritization

1: Input: minibatch k, step-size η , replay period K and size N, exponents α and β , budget T.
2: Initialize replay memory $\mathcal{H} = 0$, $\delta = 0$, $p_1 = 1$
3: Observe S_0 and choose A_0 $\sim\pi_\theta\ (S_0)$
4: for $t\ =\ 1$ **to** T **do**
5: Observe S_t, R_t, γ_t
6: Store transition $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t\ )$ in $\mathcal{H}$ with $p_t = max_{i<t}\ (p_i)$
7: If $t \equiv 0$ mod K **then**
8: for $j = 1$ to k **do**
9: Sample transition $j\sim P(j) = p_j^\alpha /\ \sum_i p_i^\alpha$
10: $w_j = \left(N.P(j)\right)^{-\beta} /\ max_i\ w_i$
11: $\delta_j = R_j + \gamma_j\ Q_{target}\left(S_j,\ \max_a Q(S_j, a)\right) - Q(S_{j-1},\ A_{j-1})$
12: $p_j\ \leftarrow |\ \delta_j|$
13: $\Delta \leftarrow \Delta + w_j.\delta_j\ .\nabla_\theta\ Q(S_{j-1},\ A_{j-1})$
14: end for
15: $\theta\ \leftarrow\ \theta + \eta.\ \Delta$,
16: $\Delta = 0$
17: $\theta_{target}\ \leftarrow\ \theta$
18: end if
19: $\boldsymbol{A_t}\sim\ \pi_\theta\ (S_t)$
20: end for

The Double DQN with Proportional Prioritization algorithm, proposed in [50] (see Algorithm 2), combines Double Q-Learning with prioritized experience replay to enhance learning efficiency in deep reinforcement learning. The algorithm begins by initializing a replay memory buffer and priority values. At each timestep, it stores observed transitions with dynamically adjusted priorities based on their temporal-difference (TD) error. Periodically, it samples minibatches using proportional prioritization $(P(j) \propto p_i^{\alpha})$ while correcting bias through importance sampling weights $(w_i = (N \cdot P(j))^{-\beta}/max\ w_i)$. The key innovation uses separate networks for action selection (main network) and value estimation (target network) to compute TD errors $\delta_j = R_j + \gamma_j\ Q_{target}\left(S_j,\ \max_a Q(S_j, a)\right) - Q(S_{j-1},\ A_{j-1})$, reducing overestimation bias. Priorities are updated as $p_i \leftarrow |\delta_i|$, and network parameters are optimized using accumulated gradients. The target network periodically synchronizes with the main network to stabilize training.

2.1.3 Deep Q-learning

Deep Q-Learning (DQL) is an off-policy reinforcement learning algorithm that uses two neural networks (NNs) to enhance training stability, the behavior Q-value function, which interacts with the environment and generates transitions $Q(s, a, \gamma, s')$, and the target $\boldsymbol{Q}$-value function, which is updated periodically and stabilizes learning, [50], [51]. At each step, the agent observes the state s, selects an action a using an ε-greedy strategy, executes the action, and stores the resulting transition in a replay buffer.

Algorithm 3: Deep Q-Learning

Begin:
-Replay memory $\mathcal{M}$
- with capacity N.
-Action-value function Q with random weights.
1: For ep = 1 to M **do**
2: $\psi_1 = \phi(s_1)$.
3:For ts =1 to T **do**
4: $a_t = \arg\max_a Q^*(\psi_1, a; \theta)$
5: $\psi_{t+1} = \phi(s_{t+1})$
6: Store transition $(\psi_1, a_t, r_t, \psi_{t+1})$ in $\mathcal{M}$
7: Sample a minibatch of transitions $(\psi_1, a_t, r_t, \psi_{t+1})$ from $\mathcal{M}$
8: If ψ_{j+1} is terminal:
9: $y_j = r_j$
10: Otherwise:
11: $y_j = r_j + \gamma\ \max_{a'} Q(\psi_{j+1}, a'; \theta)$
$\left(y_j - Q(\psi_j, a_j; \theta)\right)^2$
12.End for (ts)
13:End for (ep)

During training, the behavior network is updated by minimizing the Temporal Difference (TD) error using mini batches from the buffer. In contrast, the target network is synchronized with the behavior network at regular intervals, [52]. This approach ensures stable and efficient learning by leveraging past experiences and balancing exploration and exploitation given equation (3). The DQL algorithm proposed by [53] (see Algorithm 3) is an improvement in deep reinforcement learning that introduces a replay memory to store and reuse past experiences, reducing the high correlation between consecutive samples and improving

training stability. In each episode, the agent starts in an initial state and selects actions based on an ε policy, which balances exploration and exploitation: with probability ε, it chooses a random action to discover new strategies, while with probability 1−ε, it selects the action that maximizes the action-value function Q. Once the agent carries out the task, it earns a reward and takes note of the updated condition of the environment. Each transition $(\psi_1, a_t, r_t, \psi_{t+1})$, which contains the processed state, the chosen action, the received reward, and the next state, is stored in the replay memory $\mathcal{M}$. Instead of updating the model weights using only the latest experience, the algorithm samples a minibatch of random transitions from $\mathcal{M}$ to compute updates for the Q function. The target value y_j is determined by applying the Bellman equation, if the next state is terminal, y_j is equal to the received reward; otherwise, it sums the immediate reward r_j plus the maximum expected value of the next action, weighted by the discount factor γ. Finally, the neural network is trained by minimizing the difference between the target value y_j and the current prediction $Q(\psi_j, a_j; \theta)$ using stochastic gradient descent (SGD) or advanced optimizers like Adam. This process is repeated over multiple episodes until the Q function converges, or the maximum number of iterations is reached.

2.1.4 N-step Q-learning

The N-step Q-learning algorithm operates in a manner akin to the Deep Q-Learning (DQL), yet it incorporates several significant modifications. Unlike DQL, this algorithm does not employ a replay buffer. Instead of sampling random batches of transitions, the network undergoes training every N step, utilizing the agent's most recent N steps for this purpose. The target of the n-step Q-learning algorithm is defined by equations 5 and 6:

$$\hat{G}_{t:t+n}^{QL} = R_{t+1} + \gamma \hat{G}_{t+1:t+n}^{QL} \quad (5)$$

$$q_*(s,a) = \max_{\pi} q_\pi(s,a) \quad (6)$$

Where $\hat{G}$ is the target, n also known as the backup length parameter, R_{t+1} is the reward which are distributed according to the transition dynamics probability function known, γ is a discount factor in the interval [0,1], [54]. Unlike other Q-learning algorithms, n-step Q-learning estimates the action values corresponding to the optimal policy π^* corresponding to an optimal action-value function

$$q_*(s,a) = \max_{\pi^*} q_\pi(s,a) \quad (7)$$

The n-step Q-learning algorithm (see Algorithm 4) is an improvement over standard Q-learning that allows updating action values Q(s, a) using accumulated rewards over multiple steps instead of just one, [54] . It begins with the initialization of a Q table and a target network Q^-, along with key hyperparameters such as the learning rate α\alphaα, the discount factor γ\gammaγ, and the exploration probability ϵ\epsilonϵ.

Algorithm 4: N-steps Q-learning

Input: S, A, α, γ, ε , , I_{target}, max_{iter}
Output: $Q(s,a)$
1. Begin
2. Initialize $Q(s,a)$ arbitrarily for all $s \in$ S, a ∈ A
3. Initialize target Q-network Q^- with Q-values from Q
4. Set iteration counter T ← 0
5. repeat
6. Initialize state s_t
7. t_{start} = t
8. repeat
9. **Select action** a_t using ε-greedy policy based on $Q(s_t, a)$
11. t ← t + 1
12. until terminal s_t **or** $t - t_{start} == t_{max}$
13. if s_t is terminal **then**
14. **R ← 0**
15. else
16. R ← $\max_a Q^-(s_t, a)$
17. end if
18. **for** i from t-1 to t_start **do**
19. R ← r_i + γR
20. Q(s_i, a_i) ← Q(s_i, a_i) + α (R - Q(s_i, a_i))
21. end for
22. if T mod I_target **== 0 then**
23. **Q^- ← Q**
24. end if
25. **T ← T + 1**
26. until T > max_iter **or** training convergence reached

During each episode, the agent selects actions following an epsilon-greedy policy, balancing exploration, and exploitation, executes the action, receives a reward, and transitions to a new state. The process continues until a terminal state is reached or the maximum number of steps t_{max} is exceeded. Then, the algorithm calculates the n-step return R, incorporating the rewards obtained and using the Bellman equation to update the Q values, reducing the discrepancy between the estimate and the actual result. To improve the stability of the learning process, the target network Q− is periodically updated. Finally, the training ends when the maximum number of iterations is reached or

when the Q values converge. This approach allows for better credit assignment to future rewards, accelerates convergence, and improves training stability compared to traditional Q-learning, making it ideal for advanced reinforcement learning applications.

2.1.5 Double Q-learning

Double Q-Learning addresses the overestimation bias found in traditional Q-learning by maintaining two separate Q-value functions, denoted as Q_x and Q_y (see Algorithm 5).

Algorithm 5: Double Q-Learning

Input:
-Environment with states S, actions A, rewards R
-Hyperparameters:
α: Learning rate ($0 < \alpha \leq 1$)
γ: Discount factor ($0 \leq \gamma < 1$)
ε: Initial exploration rate ($0 \leq \varepsilon \leq 1$)
N: Maximum number of episodes
Output: Optimized Q^* tables Q^x and Q^y
Begin:
1: $Q^x(s,a) \leftarrow 0$ for all $s \in S$, $a \in A$
2: $Q^y(s,a) \leftarrow 0$ for all $s \in S$, $a \in A$
3: ε ← 1.0 (initial value)
4:For episode = 1 **to** N:
5: s ← initial state
6: **While** s is not terminal:
8: a ← random action in A
9: Otherwise:
10: $a \leftarrow \max_{a'}(Q^x(s,a') + Q^y(s,a'))$
11: If u < 0.5:
12: $a^* \leftarrow \max_{a'}(Q^x(s',a'))$
13: $Q^x(s,a) \leftarrow Q^x(s,a) + \alpha^* [r + \gamma^* Q^y(s',a^*) - Q^x(s,a)]$
14: Else:
15: $b^* \leftarrow \max_{a'}(Q^y(s',a'))$ a^*
16: $Q^y(s,a) \leftarrow Q^y(s,a) + \alpha^* [r + \gamma^* Q^x(s',b^*) - Q^y(s,a)]$
17: s ← s'
18: ε ← ε * 0.995
19: End For (episode)
Return Q^x, Q^y

Each function is updated using the value of the other function for the next state, resulting in an unbiased estimate of action values. For example, the optimal action a^* in state s' is selected based on Q_x, but $Q_y(s,a^*)$ is used to update Q_x, and vice versa. Both Q functions are trained on distinct experience samples, while actions are typically chosen using a combination of both functions, often through averaging and supplemented by ε-greedy exploration. Although this method significantly reduces overestimation bias, it does not completely resolve the challenge of accurately identifying the true maximum of expected values, as underestimations can still arise during the estimation process, [55], [56].

2.1.6 Distributional Q-learning

The core concept of distributional Q-learning is to directly engage with the complete distribution of returns rather than merely focusing on their expected value Distributional Q-learning, [57]. Let the random variable $V_{rv}(s,a)$ represent the return achieved by starting in state s executing action a, and subsequently following the current policy. Hence, given the following equation (8):

$$Q(s,a) = \mathrm{E}\{V_{rv}(s,a)\} \tag{8}$$

Unlike standard Q-learning, that find minimizes the error given by equation (1), distributional Q-learning works in the sense of minimizing a distributional error, which is the distance between full distributions,

$$\sup_{s,a} d(R_{as}(s,a) + \gamma V_{rv}(s',a^*), V_{rv}(s,a^*)) \tag{9}$$

Where R_{as} is the random variable for the immediate reward and optimal action a^* is defined by equation (10):

$$a^* = \arg\max_{a'} Q(s',a') = \arg\max_{a'} \mathrm{E}\{V_{rv}(s,a)\} \tag{10}$$

Algorithm 6: Distributional Q-learning Algorithm

Input :$x_t, a_t, r_t, x_{t+1}, \gamma_t \in [0,1]$
Output: $-\sum_i m_i \log p_i(x_t, a_t)$
Begin:
1: $Q(x_{t+1},a) := \sum_i v_i p_i(x_{t+1},a)$
2: $a^* \leftarrow \max_a \left(Q(x_{t+1,a})\right)$
3: $m_i = 0,\ i \in 0, \ldots . N-1$
4: for $j = 0$ to $N-1$ **do**
5: $\tilde{\mathcal{T}} v_j \leftarrow \left| r_t + \gamma_t v_j \right|_{V_{min}}^{V_{max}}$
6: $b_j \leftarrow \frac{(\tilde{\mathcal{T}} v_j - V_{min})}{\Delta v}$
7: $l \leftarrow \lfloor b_j \rfloor,\ \mu \leftarrow \lceil b_j \rceil$
8: $m_l \leftarrow m_l + p_j(x_t+1, a^*)(\mu - b_j)$
9: $m_\mu \leftarrow m_\mu + p_j(x_t+1, a^*)(b_j - l)$
10: end for

The Distributional Q-learning Algorithm (see algorithm 6), introduced by [57], models the full distribution of returns rather than just their expected values, providing a richer understanding of uncertainty in reinforcement learning. The algorithm begins by computing Q-values as a weighted sum over discrete value actions, then projects Bellman-

updated target values onto these atoms and distributes their probabilities via interpolation. This projection ensures values remain within predefined bounds, while the interpolation step assigns probabilities to neighboring atoms to maintain distributional continuity. The final loss (Output) minimizes the cross-entropy between the predicted and target distributions, enabling stable learning of return distributions.

Unlike the Q-learning mentioned, another approximation called Rainbow, which combines different Q-learning techniques, is proposed, where it integrates all strategies used in reinforcement learning, [58].

3 Methodologic Benchmark

The search strategy, represented in Figure 3, is based on a methodological approach like the PRISMA method [59], which includes posing the research question, literature review, elimination of duplicates, and documentation of results. With this methodology, the applications of the Q-Learning algorithm in various industries are explored to address their main challenges. The inclusion criteria require that articles be published between 2022 and 2024, contain "Q-Learning" and area-specific keywords in the title, and be published in academic journals. Reviews, conference articles (grey literature), and applications irrelevant to the area of interest are excluded. The papers were searched in databases such as Google Scholar (GS), IEEE Xplore (IEEE), and PubMed.

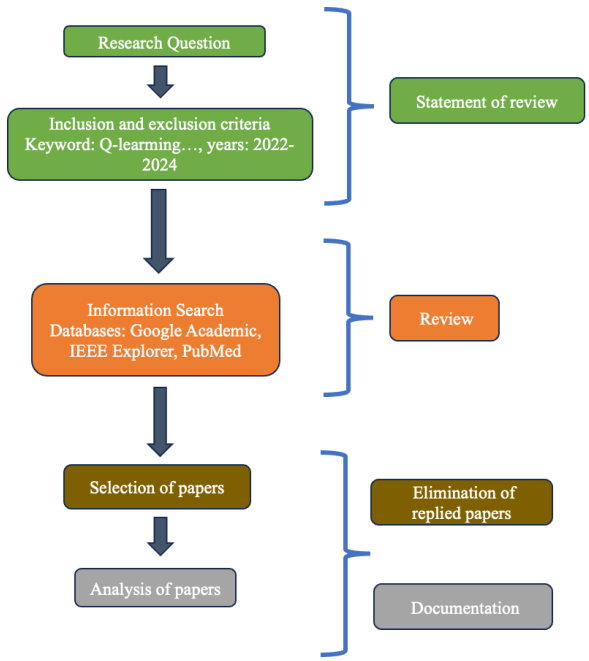

Fig. 3: Proposed Method for Systematic Review

For selecting and reviewing articles shown in Table 1, keywords (key) associated with applications in robotics, gaming, energy, finance, and medicine, identified as PC1 to PC5, respectively, were used. A hand search was performed by reviewing titles and publications in primary sources (research articles in scientific journals). Articles published in academic journals were included exclusively, and duplicates were removed. Then, the journal's quartile was determined for each selected article, and its impact factor was consulted through SJR (Scimago Journal & Country Rank).

Table 1. Search for Q-Learning in various areas for each database.

Approach	Database	Keywords (Key)
Robotics	GS	**Key1:** ("Q Learning") ("Robot" OR "Vehicle") -review -Game -Energy -Finance -Health -Medicine
	IEEE	**Key1:** ("Document Title" Q Learning) AND ("Document Title" Robot OR "Document
	PubMed	**Key1:**Q Learning [Title]) AND (Robot [Title] OR Vehicle [Title])
Games	GS	**Key2:** ("Q Learning") ("Game") -review -Robotics -Energy -Finance -Health -Medicine
	I3E	**Key2:** ("Document Title" Q Learning) AND ("Document Title" Game)
	PubMed	**Key2:**Q Learning [Title]) AND (Game [Title])
Energy	GS	**Key3:** ("Q Learning") ("Energy") -review -Robotics -Game -Finance -Health -Medicine
	IEEE	**Key3:** ("Document Title" Q Learning) AND ("Document Title" Energy)
	PubMed	**Key3:** (Q Learning [Title]) AND (Energy [Title])
Finance	GS	**Key4:** ("Q Learning") ("Fintech" OR "Trading" OR "investment" OR "Finance" OR "credit") -review -Robotics -Game -Energy -Health -Medicine
	IEEE	**Key4:** ("Document Title" Q Learning) AND ("Document Title" Fintech OR "Document Title" Trading OR "Document Title" Investment OR "Document Title" Finance OR "Document Title" Credit)
	PubMed	**Key4:** (Q Learning [Title]) AND (Fintech [Title] OR Trading [Title] OR Investment [Title] OR Finance [Title] OR Credit [Title])
Medicine	GS	**Key5:** ("Q Learning") ("health" OR "treatment") -review -Robotics -Game -Energy -Finance
	IE	**Key5:** ("Document Title" Q Learning) AND ("Document Title" health OR "Document Title" treatment)
	PubMed	**Key5:** (Q Learning [Title]) AND (health [Title] OR treatment [Title])

4 Systematic Reviews

In the database search, 434 records were found, distributed as follows: 149 for Key1, 53 for Key2, 184 for Key3, 21 for Key4, and 27 for Key5, as shown in Figure 4. The most significant number of articles was found in the Energy area, with 42.40%, followed by the Robotics area, with 34.33%, 12.21% for Games, 6.22% for Medicine, and 4.84% for Finance.

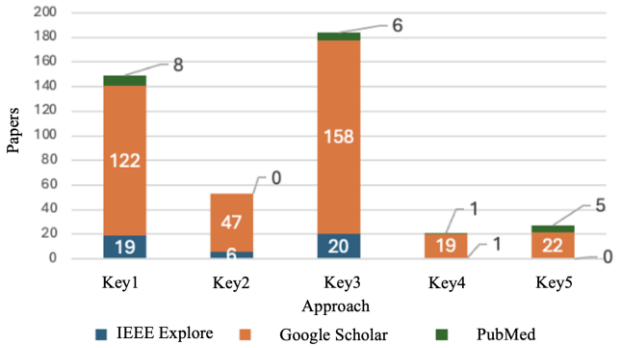

Fig. 4: Search Stage Results by Area Keywords

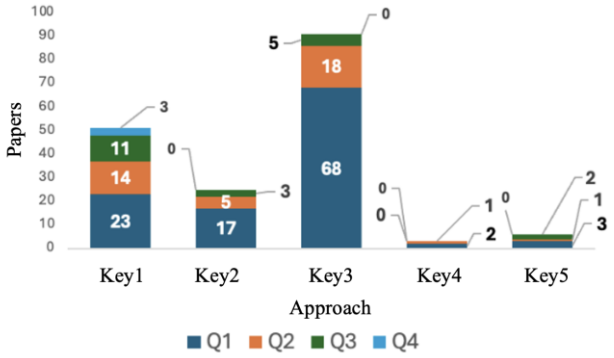

Fig. 5: Results per quartile for each area.

Of the 434 registrations obtained in the previous stage, 12.44% correspond to duplicate articles, and 46% were not selected. Next, the quartile and impact factor were identified for the remaining 176 journal articles, yielding the following distribution: 64.20% belong to Q1, 22.16% to Q2, 11.93% to Q3, and 1.70% to Q4. Figure 5 displays the results by quartile in each area according to the keywords, and Table 2 provides detailed results of the article selection.

Table 2. Results for the identification of the articles

	Q1	Q2	Q3	Q4	S	D	NS
Key1	23	14	11	3	49	27	71
Key2	17	5	3	0	25	0	28
Key3	68	18	5	0	91	20	73
Key4	2	1	0	0	3	2	16
Key5	3	1	2	0	6	5	16
Total	111	39	21	3	176	54	203

** S=Selected, D=Duplicate, NS= Not selected.*

For this review, we selected five articles described in Table 3. The letter Q represents the quartile, the abbreviation FI indicates the journal's impact factor and the research focus is determined using Q-learning.

Table 3. Characteristics of the selected items

	IF	Approach
Key1	14.4	Deep Q - Learning
Key2	11.9	fuzzy Q - Learning
Key3	14.3	meta Q - Learning
Key4	8.5	Deep Q - Learning
Key5	7.4	Deep Q-Learning

4.1 Main Findings from the Analyzed Papers

In [60], the authors propose the Q-LMPVIC model, based on Q-Learning, to improve physical collaboration between humans and robots in the industrial sector. This model uses a greedy policy that optimizes the performance of the HRC by offering an accurate and efficient solution in terms of time and computational resources. However, its effectiveness depends on the accuracy of capturing information through external sensors, which represents a limitation. The use of this reinforcement learning algorithm demonstrates significant improvements in pHRC performance. Another paper proposes a two-level reinforcement learning algorithm to achieve model-free learning for the Stackelberg game, merging the Takagi-Sugeno fuzzy logic model (T-S) and the Q-Learning technique with action-dependent Q functions. Using Q-Learning fuzzy optimal control strategies, the Stackelberg game problem for nonlinear continuous-time stochastic fuzzy systems has been solved, [61]. In [62], the authors propose a multi-objective metaheuristic based on meta-Q-learning (MQL –MM) to solve EEDFHBFSP (energy-efficient distributed fuzzy hybrid blocking flow-shop scheduling problem); it does so through a meta-training phase where it trains search operators to build the Q-Learning model and another adaptive search phase in which it performs the automatic selection of search operators.

In [63], an intraday market simulator is developed based on adaptations of the Deep Q-Networks (DQN) and Double Deep Q-Network (DDQN) algorithms within a multi-agent

architecture using recurrent neural networks (GRU). This simulator replicates the commodity futures market, including gold, using historical data. Adapting DQN and DDQN allows for training separate Q-Networks for each action, optimizing the long-term reward maximization while reducing approximation error. The results indicate significant performance and risk reduction improvements, as assessed by measures such as the Sortino Index. Additionally, the DDQN model mitigates the overestimation problems common in Q-learning models. In the work proposed in [64], the authors apply the Deep Q-Learning technique to dynamically select treatment options for patients with systemic sequential squamous cell carcinomas and predict toxicity and patient survival outcomes. They achieve this with patient-doctor digital twins, which include a patient data simulation model and treatment prescription. The results demonstrate that DQL modeling can help clinicians determine the optimal course of treatment, with a +3.73% improvement in survival when DQL treatment decisions are followed; validity could be further confirmed through a prospective clinical study. Other works focused on game environments, others on industrial optimization, while some explored medical applications. [65] tackled control problems in fuzzy stochastic systems using Q-learning within Stackelberg game frameworks. [66] addressed manufacturing challenges through their meta-Q-learning approach for multi-objective scheduling optimization. The financial domain was represented in [67], which developed a deep Q-learning system for commodity trading. Medical applications emerged in [68], employing deep Q-learning for cancer treatment optimization via digital twins. Gaming environments served as testbeds in [69] to refine Q-learning in Atari Breakout, while [70] enhanced computational aspects through parallel Q-learning implementations. This spectrum of applications - from games to life-critical systems - demonstrates Q-learning's remarkable adaptability across fundamentally different domains, each implementation tailoring the core algorithm to address unique challenges in their respective fields while contributing to the broader reinforcement learning landscape.

4.2 Tailored Q-Learning Approaches for Industrial Domains

In Table 4, we present a curated selection of Q-Learning variants optimized for distinct industrial applications, mapping each approach to its most effective use case. In human-robot collaboration (pHRC), the Q-LMPVIC model combines greedy policies with sensor integration for seamless real-time interaction. Fuzzy Q-Learning (T-S Model) proves indispensable for managing uncertainty in nonlinear control systems like Stackelberg games. The Meta-Q-Learning (MQL-MM) framework addresses complex scheduling challenges in energy and manufacturing through intelligent operator selection. Financial analysts benefit from DQN/DDQN enhanced with GRU networks, which deliver robust market forecasting capabilities. Healthcare applications leverage Deep Q-Learning (DQL) with digital twins to customize treatment plans, while Parallel Q-Learning enables scalable solutions for high-performance computing needs. For algorithm development and testing, Standard Q-Learning provides a reliable foundation in simulated environments. This comparative analysis serves as a practical reference for selecting reinforcement learning solutions aligned with specific operational demands.

Table 4. Q-Learning Approaches for Industry-Specific Applications

Q-Learning Variant	Industrial Application	Best Model/Approach	Purpose-Specific Recommendation
Q-LMPVIC	Human-Robot Collaboration (pHRC)	Greedy Policy + External Sensors	Ideal for industrial automation where real-time physical collaboration is required.
Fuzzy Q-Learning (T-S Model)	Nonlinear Control (Stackelberg Games)	Two-Level RL + Fuzzy Logic	Ideal for stochastic industrial systems requiring adaptive control under uncertainty.
Meta-Q-Learning (MQL-MM)	Energy-Efficient Scheduling (EEDFHBFSP)	Multi-Objective Metaheuristic + Operator Selection	Optimal for manufacturing/energy optimization with complex multi-objective constraints.
DQN/DDQN (GRU-Enhanced)	Commodity Trading (Financial Markets)	Multi-Agent DQN + Recurrent Networks (GRU)	Superior for financial forecasting and risk-aware trading in volatile markets.
Deep Q-Learning (DQL)	Medical Treatment Optimization	Digital Twins + Survival Outcome Prediction	Recommended for healthcare to personalize treatments (e.g., cancer therapy).
Parallel Q-Learning	High-Performance Computing (HPC)	Distributed/GPU-Accelerated Implementations	Suitable for large-scale industrial systems requiring rapid, parallelized decision-making.
Standard Q-Learning	Game Environments (e.g., Atari Breakout)	Discrete State-Action Tabling	Useful for prototyping RL algorithms in low-risk, simulated environments.

4.3 Challenges and Limitations of Q-Learning in Industrial Applications

Q-Learning faces several key challenges, particularly in complex industrial environments.

One major issue is its slow convergence, as the algorithm requires extensive exploration to learn optimal policies, making it inefficient for real-time decision-making. Additionally, high computational demands arise when dealing with large state-action spaces, especially in metaheuristic approaches where multiple search operators increase processing time. Another challenge is sensitivity to noisy sensor data, as models like Q-LMPVIC rely on accurate external inputs—any measurement errors can degrade performance. Furthermore, traditional Q-learning methods suffer from overestimation bias, which can destabilize learning, though techniques like Double Deep Q-Networks (DDQN) help mitigate this at the cost of added complexity. Finally, adapting Q-Learning to nonlinear and stochastic systems (e.g., fuzzy logic models in Stackelberg games) requires careful tuning to ensure stability and efficiency.

Despite its versatility, Q-learning has inherent limitations. Scalability becomes problematic in multi-objective optimization, where increasing the number of operators leads to exponential computational growth. The algorithm's dependence on high-quality data limits its reliability in noisy industrial settings or medical applications where sensor accuracy is critical. Moreover, while Q-Learning has been successfully applied across domains—from robotics to finance—domain-specific adaptations are often non-transferable, requiring customized solutions for each use case. Hardware constraints also pose a barrier, as advanced implementations (e.g., DQN, neuromorphic architectures) demand significant processing power, raising costs. Finally, in safety-critical applications like healthcare or autonomous systems, rigorous validation is necessary before deployment, adding time and resource burdens. Addressing these challenges will require innovations in algorithmic efficiency, hybrid learning techniques, and specialized hardware to unlock Q-Learning's full potential in real-world scenarios.

5 Conclusions

In this article, we present a preliminary review of the literature from recent years, specifically 2022-2024, focusing on reinforcement learning, mainly using the Q-learning algorithm for industrial applications. This review finds that the energy industry has seen a growing trend in research regarding the applications of Q-learning. The most widely used method in these studies is Deep Q-learning.

Future work plans to conduct a more extensive review centered around the applications of Q-learning variants, considering a wider variety of industrial sectors. The objective will be to identify the appropriate optimization parameters for each industry type based on context and specific needs characteristics.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

The authors used Grammarly to improve text clarity and language. After using this tool, the authors reviewed and edited the content as needed and took full responsibility for the content of the publication.

References:

[1] X. Wang, D. Kihara, J. Luo and G. -J. Qi, "EnAET: A Self-Trained Framework for Semi-Supervised and Supervised Learning With Ensemble Transformations," in *IEEE Transactions on Image Processing*, vol. 30, pp. 1639-1647, 2021, doi: 10.1109/TIP.2020.3044220.

[2] Y. -F. Li, L. -Z. Guo and Z. -H. Zhou, "Towards Safe Weakly Supervised Learning," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 334-346, 1 Jan. 2021, doi: 10.1109/TPAMI.2019.2922396.

[3] T. X. Pham, A. Niu, K. Zhang, T. J. T. Jin, J. W. Hong and C. D. Yoo, "Self-Supervised Visual Representation Learning via Residual Momentum," in *IEEE Access*, vol. 11, pp. 116706-116720, 2023, doi: 10.1109/ACCESS.2023.3325842.

[4] H. Sun, F. Yang and J. Ma, "Seismic Random Noise Attenuation via Self-Supervised Transfer Learning," in *IEEE Geoscience and Remote Sensing Letters*, vol. 19, pp. 1-5, 2022, Art no. 8025805, doi: 10.1109/LGRS.2022.3146173.

[5] Ge, J., Liu, Y., Gui, J., Fang, L., Lin, M., Kwok, J. T., Huang, L., & Luo, B., "Learning the Relation Between Similarity Loss and Clustering Loss in Self-Supervised Learning," in *IEEE Transactions on Image Processing*, vol. 32, pp. 3442-3454, 2023, doi: 10.1109/TIP.2023.3276708.

[6] R. Saravanan and P. Sujatha, "A State of Art Techniques on Machine Learning Algorithms: A Perspective of Supervised Learning Approaches in Data Classification," *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, 2018, pp. 945-949, doi: 10.1109/ICCONS.2018.8663155.

[7] K. P. Sinaga and M. -S. Yang, "Unsupervised K-Means Clustering Algorithm," in *IEEE Access*, vol. 8, pp. 80716-80727, 2020, doi: 10.1109/ACCESS.2020.2988796.

[8] Y. Feng, Y. Yuan, and X. Lu, "Person Reidentification via Unsupervised Cross-View Metric Learning," in *IEEE Transactions on Cybernetics*, vol. 51, no. 4, pp. 1849-1859, April 2021, doi: 10.1109/TCYB.2019.2909480.

[9] K. Nawa, E. Suryani and H. Prasetyo, "Dengue Virus Infected Leukocyte Classification on Microscopic Images with Image Histogram Based Support Vector Machine," *2019 5th International Conference on Science and Technology (ICST)*, Yogyakarta, Indonesia, 2019, pp. 1-5, doi: 10.1109/ICST47872.2019.9166385.

[10] Y. El-Zein, M. Lemay, and K. Huguenin, "PrivaTree: Collaborative Privacy-Preserving Training of Decision Trees on Biomedical Data," in *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 21, no. 1, pp. 1-13, Jan.-Feb. 2024, doi: 10.1109/TCBB.2023.3286274.

[11] S. Jeong and J. Lee, "Modulation Code and Multilayer Perceptron Decoding for Bit-Patterned Media Recording," in *IEEE Magnetics Letters*, vol. 11, pp. 1-5, 2020, Art no. 6502705, doi: 10.1109/LMAG.2020.2993000.

[12] J. Vieira, R. P. Duarte and H. C. Neto, "kNN-STUFF: kNN Streaming Unit for Fpgas," in *IEEE Access*, vol. 7, pp. 170864-170877, 2019, doi: 10.1109/ACCESS.2019.2955864

[13] M. S. Mahmud, J. Z. Huang, R. Ruby, A. Ngueilbaye and K. Wu, "Approximate Clustering Ensemble Method for Big Data," in IEEE Transactions on Big Data, vol. 9, no. 4, pp. 1142-1155, 1 Aug. 2023, doi: 10.1109/TBDATA.2023.3255003.

[14] N. A. Mat Isa, S. A. Salamah, and U. K. Ngah, "Adaptive fuzzy moving K-means clustering algorithm for image segmentation," in *IEEE Transactions on Consumer Electronics*, vol. 55, no. 4, pp. 2145-2153, November 2009, doi: 10.1109/TCE.2009.5373781.

[15] C. Zheng, C. Hu, Y. Chen and J. Li, "A Self-Learning-Update CNN Model for Semantic Segmentation of Remote Sensing Images," in *IEEE Geoscience and Remote Sensing Letters*, vol. 20, pp. 1-5, 2023, Art no. 6004105, doi: 10.1109/LGRS.2023.3261402.

[16] T. Joseph and T. S. Bindiya, "Realization and Hardware Implementation of Gating Units for Long Short-Term Memory Network Using Hyperbolic Sine Functions," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 5141-5145, Dec. 2023, doi: 10.1109/TCAD.2023.3293045

[17] A. Dash, J. Ye, and G. Wang, "A Review of Generative Adversarial Networks (GANs) and Its Applications in a Wide Variety of Disciplines: From Medical to Remote Sensing," in *IEEE Access*, vol. 12, pp. 18330-18357, 2024, doi: 10.1109/ACCESS.2023.3346273.

[18] Saeed, U., Shah, S. Y., Alotaibi, A. A., Althobaiti, T., Ramzan, N., Abbasi, Q. H., & Shah, S.A., "Portable UWB RADAR Sensing System for Transforming Subtle Chest Movement Into Actionable Micro-Doppler Signatures to Extract Respiratory Rate Exploiting ResNet Algorithm," in *IEEE Sensors Journal*, vol. 21, no. 20, pp. 23518-23526, 15 Oct.15, 2021, doi: 10.1109/JSEN.2021.3110367.

[19] Zhu, Z., Zhai, W., Liu, H., Geng, J., Zhou, M., Ji, C., & Jia, G., "Juggler-ResNet: A Flexible and High-Speed ResNet Optimization Method for Intrusion Detection System in Software-Defined Industrial Networks," in *IEEE Transactions on Industrial Informatics*, vol. 18, no. 6, pp. 4224-4233, June 2022, doi: 10.1109/TII.2021.3121783.

[20] Z. Yu, H. Wang, D. Wang, Z. Li, and H. Song, "CGFuzzer: A Fuzzing Approach Based on Coverage-Guided Generative Adversarial Networks for Industrial IoT Protocols," in *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21607-21619, 1 Nov.1, 2022, doi: 10.1109/JIOT.2022.3183952.

[21] R. V. Godoy, A. Dwivedi, B. Guan, A. Turner, D. Shieff and M. Liarokapis, "On EMG Based Dexterous Robotic Telemanipulation: Assessing Machine Learning Techniques, Feature Extraction Methods, and Shared Control Schemes," in *IEEE Access*, vol. 10, pp. 99661-99674, 2022, doi: 10.1109/ACCESS.2022.3206436.

[22] M. Tufail, J. Iqbal, M. I. Tiwana, M. S. Alam, Z. A. Khan, and M. T. Khan, "Identification of Tobacco Crop Based on Machine Learning for a Precision Agricultural Sprayer," in *IEEE Access*, vol. 9, pp. 23814-23825, 2021, doi: 10.1109/ACCESS.2021.3056577.

[23] Zhong, Y., Zhang, S., Liu, Z., Zhang, X., Mo, Z., Zhang, Y., Hu, H., Chen, W., & Qi, L., "Unsupervised Fusion of Misaligned PAT and MRI Images via Mutually Reinforcing Cross-Modality Image Generation and Registration," in *IEEE Transactions on Medical Imaging*, vol. 43, no. 5, pp. 1702-1714, May 2024, doi: 10.1109/TMI.2023.3347511.

[24] M. R. Ahmed, Y. Zhang, Z. Feng, B. Lo, O. T. Inan and H. Liao, "Neuroimaging and Machine Learning for Dementia Diagnosis: Recent Advancements and Future Prospects," in *IEEE Reviews in Biomedical Engineering*, vol. 12, pp. 19-33, 2019, doi: 10.1109/RBME.2018.2886237.

[25] M. Bahrami and M. Forouzanfar, "Sleep Apnea Detection From Single-Lead ECG: A Comprehensive Analysis of Machine Learning and Deep Learning Algorithms," in *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1-11, 2022, Art no. 4003011, doi 10.1109/TIM.2022.3151947.

[26] C. Lastre-Domínguez, Y. S. Shmaliy, O. Ibarra-Manzano, and M. Vazquez-Olguin, "Denoising and Features Extraction of ECG Signals in State Space Using Unbiased FIR Smoothing," in *IEEE Access*, vol. 7, pp. 152166-152178, 2019, doi: 10.1109/ACCESS.2019.2948067.

[27] J. Yan, J., Hu, L., Zhen, Z., Wang, F., Qiu, G., Li, Y., Yao, L., Shafie-Khah, M., & Catalao, J. P. S.., "Frequency-Domain Decomposition and Deep Learning Based Solar PV Power Ultra-Short-Term Forecasting Model," in *IEEE Transactions on Industry Applications*, vol. 57, no. 4, pp. 3282-3295, July-Aug. 2021, doi: 10.1109/TIA.2021.3073652.

[28] Z. Yang, X. Peng, J. Song, R. Duan, Y. Jiang and S. Liu, "Short-Term Wind Power Prediction Based on Multi-Parameters Similarity Wind Process Matching and Weighed-Voting-Based Deep Learning Model Selection," in *IEEE Transactions on Power Systems*, vol. 39, no. 1, pp. 2129-2142, Jan. 2024, doi: 10.1109/TPWRS.2023.3257368.

[29] R. S. Sutton y A. G. Barto, "Introduction," in Reinforcement Learning: An Introduction, 2da ed. Cambridge, Massachusetts, EE.UU.: The MIT Press, 2018, capítulo 1, pp. 25-42. ISBN: 9780262039246.

[30] Hala Khankhour, Chakir Tajani, Najat Rafalia, Jaafar Abouchabaka, "Robotic Agents through Scalable Multi-agent Reinforcement Learning for Optimization of Warehouse Logistics," WSEAS Transactions on Systems and Control, vol. 20, pp. 42-49, 2025, doi: 10.37394/23203.2025.20.5

[31] Ashraf A. Abu-Ein, Waleed Abdelkarim Abuain, Mohannad Q. Alhafnawi, Obaida M. Al-Hazaimeh, "Security Enhanced Dynamic Bandwidth Allocation-Based Reinforcement Learning," *WSEAS Transactions on Information Science and Applications,* vol. 22, pp. 21-27, 2025, https://doi.org/10.37394/23209.2025.22.3.

[32] Ricardo Yauri, Frank Silva, Ademir Huaccho, Oscar Llerena, "Intelligent Traffic Light System using Deep Reinforcement Learning," *WSEAS Transactions on Systems and Control,* vol. 18, pp. 263-271, 2023, https://doi.org/10.37394/23203.2023.18.26.

[33] Daschievici Luiza, Ghelase Daniela, "The Cognitive Engineering in Manufacturing Process through Reinforcement Learning," *WSEAS Transactions on Systems,* vol. 22, pp. 194-198, 2023, https://doi.org/10.37394/23202.2023.22.19.

[34] [34] S. Muthurajan, Rajaji Loganathan, R. Rani Hemamalini, "Deep Reinforcement Learning Algorithm based PMSM Motor Control for Energy Management of Hybrid Electric Vehicles," *WSEAS Transactions on Power Systems,* vol. 18, pp. 18-25, 2023, https://doi.org/10.37394/232016.2023.18.3.

[35] Mahmoud Almasri, Xavier Marjou, Fanny Parzysz, "Reinforcement-Learning Based Handover Optimization for Cellular UAVs Connectivity," W*SEAS Transactions on Computer Research,* vol. 10, pp. 93-98, 2022, https://doi.org/10.37394/232018.2022.10.12.

[36] Hitesh Shah, M.Gopal, "A Reinforcement Learning Algorithm With Evolving Fuzzy Neural Networks," *International Journal of Electrical Engineering and Computer Science,* vol. 2, pp. 68-72, 2020.

[37] Dinh-Son Vu, Ahmad Alsmadi, "Trajectory Planning of a CableBased Parallel Robot using Reinforcement Learning and Soft Actor-Critic," *WSEAS Transactions on Applied and Theoretical Mechanics,* vol. 15, pp. 165-172, 2020, https://doi.org/10.37394/232011.2020.15.19.

[38] Junyi Mo, Shunichi Ohmori, "Crowd Sourcing Dynamic Pickup & Delivery Problem considering Task Buffering and Drivers' Rejection -Application of Multi-agent Reinforcement Learning-," *WSEAS Transactions on Business and Economics,* vol. 18, pp. 636-645, 2021, https://doi.org/10.37394/23207.2021.18.63.

[39] K. Sivamayil, E. Rajasekar, B. Aljafari, S. Nikolovski, S. Vairavasundaram, and I. Vairavasundaram, "A Systematic Study on Reinforcement Learning Based Applications," **Energies**, vol. 13, no. 9886, pp. 1-23, Aug. 2020. doi: 10.3390/en16031512.

[40] Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning, and teaching. *Machine Learning,* 8(3-4), 293–321. doi: 10.1007/bf00992699.

[41] B. Jang, M. Kim, G. Harerimana, and J. W. Kim, "Q-Learning Algorithms: A Comprehensive Classification and Applications," *IEEE Access,* vol. 7, no. 8836506, pp. 133653–133667, Sep. 2019. doi: 10.1109/ACCESS.2019.2941229.

[42] Watkins, C. J. C. H., & Dayan, P. (1992). *Technical Note: Q-Learning. Machine Learning,* 8(3-4), 279-292. © 1992 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

[43] Clifton, Jesse, and Eric Laber. "Q-Learning: Theory and Applications." *Annual Review of Statistics and Its Application* 7 (2020): 279–301. https://doi.org/10.1146/annurev-statistics-031219-041220.

[44] Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. *In Proceedings of the 12th International Conference on Neural Information Processing Systems* (NIPS'99), *Part of Advances in Neural Information Processing System*. MIT Press, Cambridge, MA, USA, Nov, 1999, pp.1057–1063.

[45] Moerland, T. M., Broekens, J., Plaat, A., & Jonker, C. M. (2023). Model-based Reinforcement Learning: A Survey. arXiv preprint arXiv:2006.16712.

[46] Dann, C., Mansour, Y., Mohri, M., Sekhari, A., & Sridharan, K., (2022, July). "Guarantees for epsilon-greedy reinforcement learning with function approximation." *39th International conference on machine learning, (PMLR, 2022),* Baltimore, Maryland, USA; pp. 4666-4689.

[47] A. Mignon, R.L.A. da Rocha (2017). An Adaptive Implementation of ε-Greedy in Reinforcement Learning. *Recent Advances in Computer Engineering Series,* Vol.109, pp.1146-1151.

[48] Tokic, M. Adaptive ε-Greedy Exploration in Reinforcement Learning Based on Value Differences. In: Dillmann, R., Beyerer, J., Hanebeck, U.D., Schultz, T. (eds) KI 2010: Advances in Artificial Intelligence. KI 2010. *Lecture Notes in Computer Science,* vol 6359. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-16111-7_23.

[49] W. Yuan, Y. Li, H. Zhuang, C. Wang, and M. Yang, "Prioritized Experience Replay-Based Deep Q Learning: Multiple-Reward Architecture for Highway Driving Decision Making," in *IEEE Robotics & Automation Magazine*, vol. 28, no. 4, pp. 21-31, Dec. 2021, doi: 10.1109/MRA.2021.3115980.

[50] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. arXiv preprint arXiv:1511.05952.

[51] Wang, Z., Schaul, T., Hessel, M., Van Hasselt, H., Lanctot, M., & De Freitas, N. (2016, June). Dueling Network Architectures for Deep Reinforcement Learning. Proceedings of the 33rd International Conference on Machine Learning. IMCL´16, New York, USA, vol 48, pp. 1995-2003, [Online]. https://proceedings.mlr.press/v48/wangf16.html (Accessed Date: October 14, 2024).

[52] Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra and Martin A. Riedmiller. "Playing Atari with Deep Reinforcement Learning." ArXiv abs/1312.5602 (2013), 1-9.

[53] N. Hariharan & G, Paavai. (2022). A Brief Study of Deep Reinforcement Learning with Epsilon-Greedy Exploration. International *Journal of Computing and Digital Systems.* vol.11, pp.541-551. doi: 10.12785/ijcds/110144.

[54] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver and Koray Kavukcuoglu. "Asynchronous Methods for Deep Reinforcement Learning." Proceedings 33rd *International Conference on Machine Learning* (2016). N-steps. New York, USA, vol. 48, pp 1928-1937.

[55] Van Hasselt, Hado. "Double Q-Learning*." In Advances in Neural Information Processing Systems 23: 24th Annual Conference on Neural Information Processing Systems 2010, 2613–2621.* Proceedings of a meeting held December 6–9, 2010, Vancouver, British Columbia, Canada. 2010.

[56] Hernandez-Garcia, J. Fernando, and Richard S. Sutton. "Understanding multi-step deep reinforcement learning: A systematic study of the DQN target." *arXiv preprint arXiv:1901.07510* (2019).

[57] Bellemare, Marc G., Will Dabney, and Rémi Munos. "A distributional perspective on reinforcement learning." Proceedings of the 34th *International conference on machine learning*. PMLR, 2017. Sydney NSW Australia, vol. 70, pp 449-458.

[58] Hessel, M., Modayil, J., Van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., &

Silver, D. (2017). Rainbow: Combining Improvements in Deep Reinforcement Learning. arXiv preprint arXiv:1710.02298.

[59] PRISMA. (2024, April 25). Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA), [Online]. https://prisma-statement.org (Accessed Date: October 14, 2024).

[60] Wang, R., Zhuang, Z., Tao, H., Paszke, W., & Stojanovic, V., "Q-learning based fault estimation and fault tolerant iterative learning control for MIMO systems." *ISA transactions*, 142 (2023): 123-135.

[61] Tresca, Luigi, Luca Pulvirenti, Luciano Rolando, and Federico Millo. "Development of a Deep Q-Learning Energy Management System for a Hybrid Electric Vehicle." Transportation Engineering 16 (2024): 100241. ISSN: 2666-691X. https://doi.org/10.1016/j.treng.2024.100241.

[62] Correa, Arthur, Alexandre Jesus, Cristóvao Silva, Paulo Peças, and Samuel Moniz. "Rainbow Versus Deep Q-Network: A Reinforcement Learning Comparison on the Flexible Job-Shop Problem." *IFAC-PapersOnLine* 58, no. 19 (2024): 870–875. ISSN: 2405-8963. https://doi.org/10.1016/j.ifacol.2024.09.176.

[63] Van Hasselt, Hado, Arthur Guez, and David Silver. "Deep reinforcement learning with double q-learning." *Proceedings of the AAAI´16 of the Thirtieth conference on artificial intelligence*. Phoenix Arizona. Vol. 30. No. 1.pp 2049-2100, 2016.

[64] J. Wang, K. Zhu, P. Dai, and Z. Han, "An Adaptive Q-Value Adjustment-Based Learning Model for Reliable Vehicle-to-UAV Computation Offloading," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 5, pp. 3699–3713, Oct. 2023. doi: 10.1109/TITS.2023.3322748.

[65] Z. Ming, H. Zhang, Y. Yan, and L. Yang, "Adaptive Optimal Control via Q-Learning for Itô Fuzzy Stochastic Nonlinear Continuous-Time Systems With Stackelberg Game," *IEEE Trans. Fuzzy Syst.*, vol. 32, no. 4, pp. 2029-2038, Mar. 2024, doi: 10.1109/TFUZZ.2023.3344123.

[66] Z. Shao, W. Shao, J. Chen, and D. Pi, "MQL-MM: A Meta-Q-Learning-Based Multi-Objective Metaheuristic for Energy-Efficient Distributed Fuzzy Hybrid Blocking Flow-Shop Scheduling Problem," *IEEE Trans. Evol. Comput.*, pp. 1-1, May 2024, doi: 10.1109/TEVC.2024.3399314.

[67] M. Massahi and M. Mahootchi, "A deep Q-learning based algorithmic trading system for commodity futures markets," Expert Systems with Applications, pp. 1-15, Mar. 2024. https://doi.org/10.1016/j.eswa.2023.121711.

[68] E. Tardini, X. Zhang, G. Canahuate, A. Wentzel, A. S. R. Mohamed, L. Van Dijk, C. D. Fuller, and G. E. Marai, "Optimal treatment selection in sequential systemic and locoregional therapy of oropharyngeal squamous carcinomas: deep Q-learning with a patient-physician digital twin dyad," *Journal of Medical Internet Research*, vol. 24, no. 4, pp. 1–21, 2022. https://doi.org/10.2196/29455.

[69] Fernández-Vizcaíno, G., & Gallego-Durán, F. J. Ajustando. Q-Learning for generating automatic players: an example based on Atari Breakout. Santander-UA Chair of Digital Transformation, University of Alicante (Q-Learning para generar jugadores automáticos: un ejemplo basado en Atari Breakout. Cátedra Santander-UA de Transformación Digital, Universidad de Alicante).

[70] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518, 529–533 (2015). doi: 10.1038/nature14236.

Contribution of Individual Authors to the Creation of a Scientific Article (Ghostwriting Policy)

- Guillermina Vivar Estudillo led the research project and supervised the research, paper preparation, review, and editing.
- María Estudillo-Ayala, José Beltrán-Hernández, and Mario Ibarra-Manzano contributed to the work methodology.
- Carlos Lastre-Domínguez was responsible as the corresponding author and project manager.

Sources of Funding for Research Presented in a Scientific Article or Scientific Article Itself

No funding was received to conduct this study.

Conflict of Interest

The authors have no conflicts of interest to declare.